

F-TALENT CODE 2026**Valid Parenthesis Expression**

Time limit: 1.00 s | Memory limit: 512 MB

```
{ } [ ] ( ) < >
{ } [ ] ( ) < >
{ } [ ] ( ) < >
```

Problem Statement

Your consulting firm has been asked by a client to invent a compact encoding for every valid parenthesis string of even length n . A string is valid when it obeys the usual recursive rules:

“() ” is valid;

if S is valid then so is “(S)”;

if S and T are valid then their concatenation ST is also valid.

A colleague proposes the following generation scheme: begin with any single valid string of length n and, at every subsequent step, invert exactly two parentheses. All later strings can therefore be described simply by the pair of positions that were flipped relative to the preceding string. It is known that a walk of this kind exists that visits every valid parenthesis string of length n exactly once.

Your task is to produce one such walk.

Input

The only line of the input contains a single integer n .

Output

On the first line of the output, output your starting parenthesis expression.

On the consecutive lines, output two integers describing the indices of the parentheses to be flipped.

Constraints

$2 \leq n \leq 20$

n is even

Example 1:

INPUT	OUTPUT
6	((())) 3 4 2 3 4 5 2 3

Explanation: The operations correspond to the following parenthesis sequences:

((()))

((00))

(0(0))

(000)

(0)0